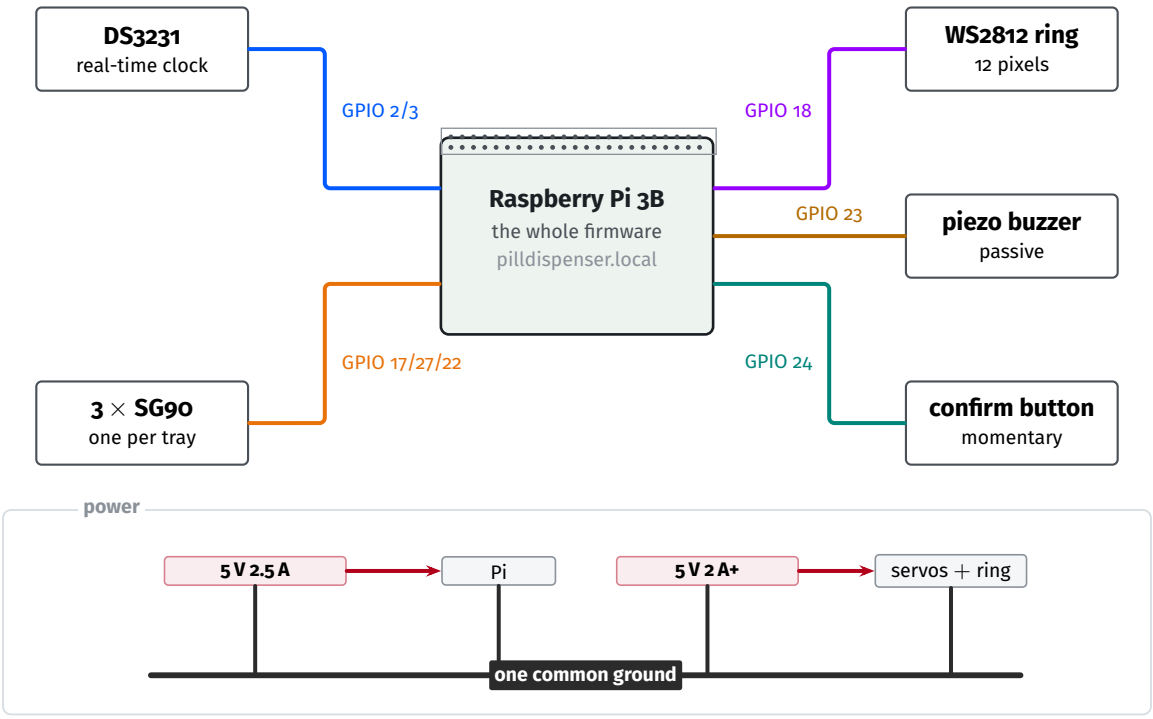
# Building the dispenser

A bare Raspberry Pi 3B to a machine that drops a pill and chirps about it

SnapCare · Team H1 · 16th e-ICON World Contest




Eleven stages. Each one adds a single subsystem
and ends with the command that proves it works.

Matches `snapcare_dispenser/config.py`. Where the two disagree, that file is right.

Generated from `docs/assembly-guide.tex`.

## Contents

1 The build at a glance 3

2 Bill of materials 3

3 The header, and every pin this build touches 5

4 Stage 0 — the Pi, before anything is wired to it 6

5 Stage 1 — the DS3231 6

6 Stage 2 — the confirm button 7

7 Stage 3 — the buzzer 8

8 Stage 4 — the LED ring 8

9 Stage 5 — the servo power rail, before any servo 9

10 Stage 6 — one servo 9

11 Stage 7 — the other two servos 10

12 Stage 8 — mechanical fitting and trim 11

13 Stage 9 — cloud sync, if you want it 12

14 Stage 10 — the label 14

15 The whole pin map, in one place 15

16 Power budget 15

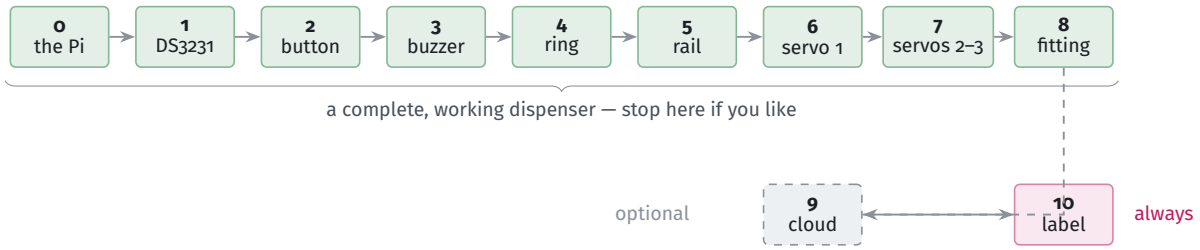
17 Commissioning checklist 16

18 When it does not work 17

**Read this first.** Assemble in the order given and test after every stage. Each stage adds one subsystem and ends with a command that proves it. Wiring the whole thing and then powering it on means debugging five subsystems at once — and on this device three of the five fail *silently*. A servo on the wrong pin, a ring that never lights, and an RTC with a dead cell all look exactly the same from the outside: a machine sitting there doing nothing.

## 1 The build at a glance

Stages 0 to 8 build the machine. Stage 9 is optional — it connects the finished machine to the app over the internet — and stage 10 is the label. If you only want a working dispenser, stop after stage 8 and print the label: nothing in stages 0 to 8 depends on there being a network beyond the house.




| | Subsystem | Where it lands | What it is for |
|---|---|---|---|
| | DS3231 | GPIO 2/3 ($I^2C$) + 3.3 V | knows the time after a power cut |
| | WS2812 ring | GPIO 18, through 330 Ω | green / blue / amber / violet |
| | piezo buzzer | GPIO 23, through 100 Ω | the sound that fetches a person from another room |
| | confirm button | GPIO 24 to GND | internal pull-up, 20 ms glitch filter |
| | 3 × SG90 | GPIO 17, 27, 22 | signal only — never power |

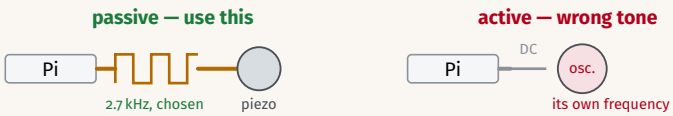
## 2 Bill of materials

| | Qty | Part | Notes |
|---|---|---|---|
|  | 1 | Raspberry Pi 3B | Any Pi with the 40-pin header works; the pin map assumes a 3B |
|  | 1 | microSD card, 8 GB+ | Raspberry Pi OS **Bookworm or newer** — the firmware drives WiFi through NetworkManager |
|  | 1 | Pi power supply, 5 V 2.5 A | The official one. Servos do **not** come off this |
|  | 3 | SG90 micro servo | One per fitted tray. The firmware handles eight |
|  | 1 | 5 V supply, 2 A or better | Separate, for the servos and the ring |
|  | 1 | 1000 µF electrolytic, 10 V+ | Across the servo rail, physically close to the servos |
|  | 1 | WS2812 / NeoPixel ring, 12 px | 16 and 24 also work — set `RING_PIXELS` |
|  | 1 | 330 Ω resistor | In the ring's data line, at the **ring** end |

| | Qty | Part | Notes |
|---|---|---|---|
|  | 1 | Passive piezo buzzer | Passive, not an active beeper — see below |
|  | 1 | 100 Ω resistor | In series with the buzzer |
|  | 1 | Momentary push button | Any normally-open button. No external resistor needed |
|  | 1 | DS3231 RTC module | With a coin cell fitted. Read the ZS-042 warning below |
|  | — | Jumper wire, terminal blocks | Dupont leads are fine for the bench; solder or screw terminals for anything that gets carried |
|  | — | a relay, if you want cloud sync | *Not a part.* One container, deployed once for all your devices — see `../cloud_relay/`. Skip it and the dispenser is LAN-only |

### "Passive" buzzer

The firmware makes the tone itself — pigpio drives the pin as a 2.7 kHz square wave (`indicators.Buzzer.beep`). An *active* buzzer contains its own oscillator and sounds at its own frequency the moment it sees a voltage, so it will still make a noise here, just not the one that was chosen. 2.7 kHz is where a small piezo is loudest and where age-related hearing loss has bitten least, which is the whole reason the number is in `config.py`.




### The DS3231 module and its coin cell

The commonly sold ZS-042 board has a charging circuit intended for a rechargeable LIR2032. Fitted with an ordinary non-rechargeable CR2032 — which is what most of them ship next to — it trickles current into a cell that cannot take it. The usual fix is to remove the series resistor in that charging path (marked `201` on most boards) or to use an LIR2032. Either way, **fit the cell before the first `hwclock --systohc`**, because the whole point of the chip is what it remembers with the power off.

## 3 The header, and every pin this build touches

Keep this page open while you wire. Everything is BCM numbering, which is what pigpio and RPi.GPIO both use; the small number inside each pad is the *physical* pin, which is what you actually count on the board.




### The pins that were avoided, and what each costs

| Pin | Why it is not used |
|---|---|
| GPIO 0, 1 | HAT ID EEPROM. Driving them is how a HAT you plug in later fails to be detected |
| GPIO 2, 3 | Permanent 1.8 kΩ $I^2C$ pull-ups — a servo line idling high slams the horn at power-on. Now the DS3231 lives here |
| GPIO 14, 15 | The serial console: the one way back into a Pi whose network setup you just broke |
| GPIO 7–11 | SPI. Free today, gone the moment anyone adds an SPI display |

#### Reserved for trays 4–8

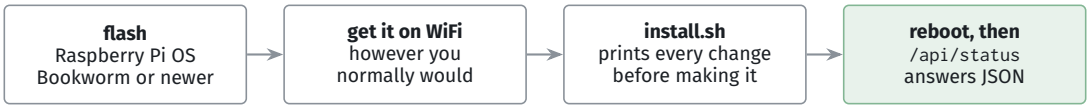
GPIO 5, 6, 13, 19 and 26 are already checked for clashes and for the pins that cost hardware, so a fourth hopper is a plug and a number, not a re-derivation:

```
SNAPCARE_TRAYS=8 python3 -m snapcare_dispenser
```

`tests/test_servo.py` asserts all of this, and that no peripheral shares a pin with a servo.

## 4 Stage 0 — the Pi, before anything is wired to it

Do this stage first because it is the only one that can be done comfortably with the Pi on a desk and a keyboard attached. Once the servos are on it, every mistake costs a power-down.




```
git clone <this repo> && cd h1-SnapCare/pill_dispenser/rpi_firmware
sudo ./service/install.sh
sudo reboot            # the installer will tell you if the I2C overlay needs one
```

The installer is safe to re-run. It installs pigpio and enables `pigpiod`, sets the hostname to `pilldispenser` and publishes it over avahi, writes the captive-portal DNS rule, turns on I²C with the `i2c-rtc,ds3231` overlay, disables `fake-hwclock`, installs `python3-cryptography` for cloud sync, and installs the systemd unit.

### Prove it

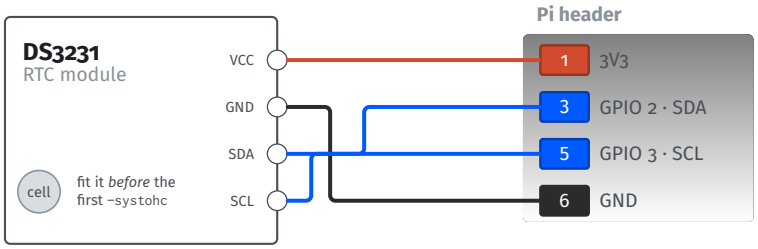
```
systemctl status snapcare-dispenser
curl -s http://localhost/api/status
```

You should get JSON. It will say the clock is untrusted and that no servo hardware is present — both correct, nothing is wired yet. The `cloud` object will say `unpaired`, which is also correct: no relay has been named.

## 5 Stage 1 — the DS3231

**Power off first — every time**

Not “put it to sleep”. Unplug it. Every wiring stage in this guide starts that way, and this reminder is not repeated after this page.




**3.3 V, not 5 V.** The module runs happily at either, but at 5 V its pull-ups sit on the Pi’s I²C lines at 5 V, and those pins are not 5 V tolerant.

**No pull-up resistors.** GPIO 2/3 carry permanent 1.8 kΩ pull-ups on the board, and the module has its own.

Physical pin numbers are in the pads. For where each one sits on the board, see the header map on page 5.

That permanent pull-up is also exactly why GPIO 2 and 3 are kept out of the servo map: a servo signal line that idles high before the software starts slams the horn to one end at power-on, with a loaded tray on it.

**Prove it**

```
sudo i2cdetect -y 1        # expect 68 (and often 57, the EEPROM beside it)
ls -l /dev/rtc0            # the kernel overlay claimed it
sudo hwclock -r            # what the chip believes
sudo hwclock --systohc     # once the Pi's own time is right, write it in
```

Then confirm the firmware agrees the clock can be *believed*, which is a different question from whether the chip is there:

```
curl -s http://localhost/api/status     # the "clock" object: trusted, source, rtc
```

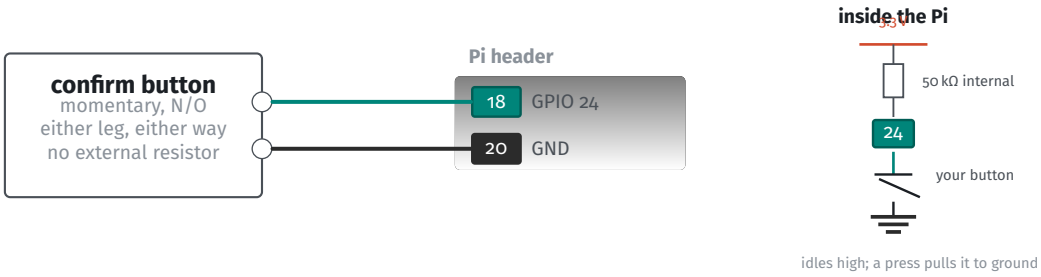
**A dead cell does not make the chip vanish from the bus**

It makes it answer promptly with nonsense. The firmware reads the oscillator-stop flag (bit 7 of register `0x0F`) and only a clear flag counts. If it reads `untrusted` here with a good cell fitted, set the time once through the API and it will clear the flag in the right order:

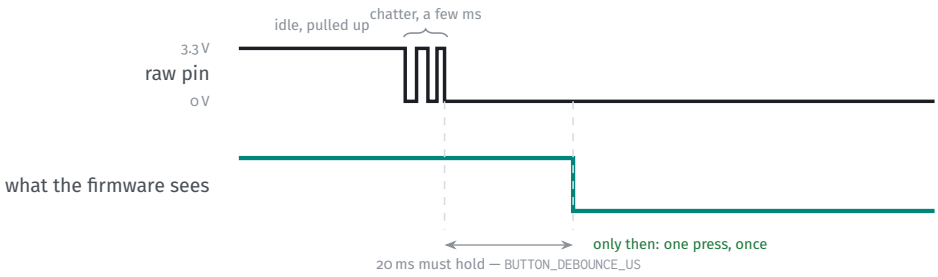
```
curl -X POST http://localhost/api/clock -d '{"iso":"2026-07-29T09:15:00"}'
```

## 6 Stage 2 — the confirm button

Two wires, nothing else.




**Why there is no debounce capacitor.** A real switch does not close once; it chatters for a few milliseconds. Pigpio's glitch filter discards any edge that does not then hold for 20 ms (`BUTTON_DEBOUNCE_US`), *before* a callback is ever queued. Undebounced, one press records several doses.




**Prove it**

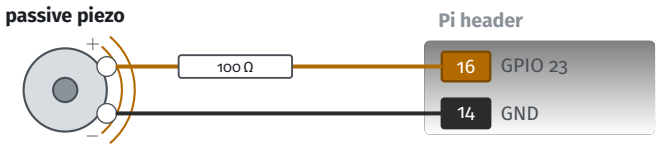
You need a dose waiting to confirm, so this is easiest tested at the end of Stage 3 — but you

can check the firmware claimed the pin now:

```
journalctl -u snapcare-dispenser | grep -i button
```

It should report the `pigpio` backend. `poll` means pigpiod is not running, and `none` means neither is available.

## 7 Stage 3 — the buzzer




Any free GPIO would do — the tone is software-timed PWM from pigpio, so it does not compete with the ring for a hardware PWM channel.

The tone is **2.7 kHz**, 0.12 s, repeating every 20 s while a dose is waiting.

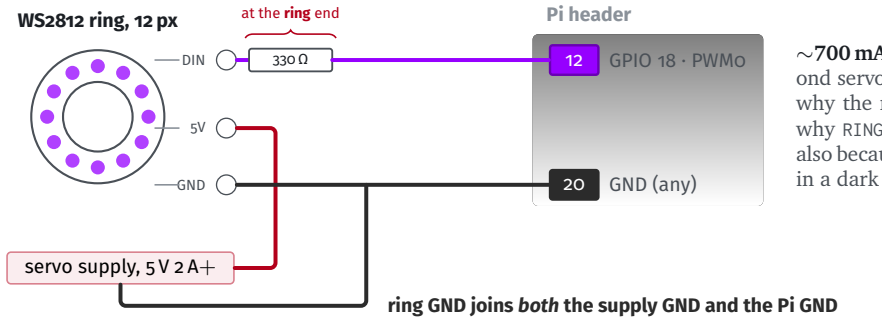
### Prove it

```
curl -X POST http://localhost/api/drop -d '{"tray":1}'
```

With nothing else wired this drops a tray that does not exist yet, which is fine — what you are listening for is the chirp that starts when the dose goes into the confirm state, repeating every 20 s. **Press the button to stop it**, which tests Stage 2 at the same time. If it will not stop, the button is the problem; if nothing sounds, the buzzer is.

## 8 Stage 4 — the LED ring

This is the first stage that touches the servo supply. The ring's 5 V does *not* come from the Pi.




**~700 mA at full white.** That is a second servo's worth of current, which is why the ring is on the servo rail and why `RING_BRIGHTNESS` is 60 of 255 — also because full brightness is dazzling in a dark bedroom at 3 a.m.

### GPIO 18 is not a free choice

`rpi_ws281x` generates the WS2812's 800 kHz waveform with the PWM or PCM peripheral and the DMA controller, and **GPIO 18, 12, 21 and 10 are the only pins wired to them.** Anything else is a ring that never lights and an error message about DMA. GPIO 18 is PWM0 and is not in the servo map. A test asserts this (`tests/test_servo.py:181`).

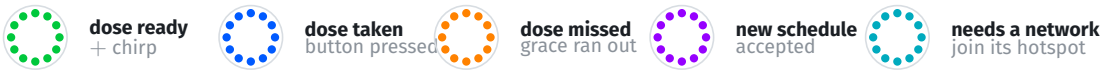
**The 330 Ω is not folklore.** It damps the reflection on the first data edge, which is what makes the first pixel latch a wrong colour on a long lead. It has to be at the ring end to do that.

**Prove it**

```
sudo systemctl restart snapcare-dispenser
curl -s http://localhost/api/status     # "indicators" should now list the ring
curl -X POST http://localhost/api/schedule \
     -H 'Content-Type: application/json' \
     -d '{"payload":"SC1*W*1:0800-127-030"}'   # violet: schedule accepted
```

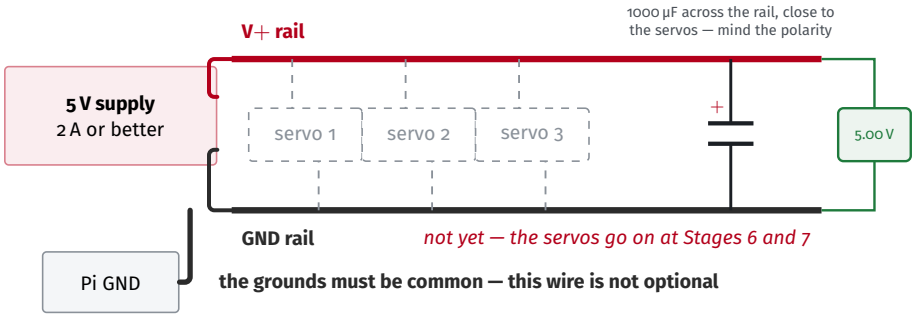
If the ring stays dark, check `journalctl` — a ring that throws is dropped and logged rather than retried forever, and the status page reports what is actually fitted rather than pretending.

**The colour vocabulary you have just fitted.** Never text, because the person the machine is for cannot be assumed to read.




## 9 Stage 5 — the servo power rail, before any servo

Wire and measure the rail with **no servos attached and no signal wires to the Pi**. This is the stage that protects the SD card.




**Do not run the servos from the Pi's 5 V pin**

One SG90 stalls at around 700 mA. The failure is not a blown regulator; it is a brown-out, and **a Pi that browns out mid-write corrupts an SD card** — where the file being written is the one that records whether a dose has already been given.

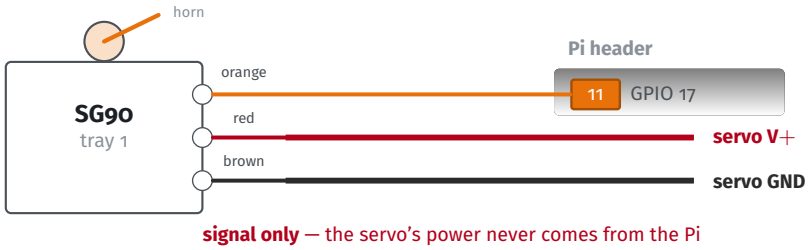
The common ground is what makes the signal mean anything. Without it the servos twitch or ignore commands entirely, which reads as a firmware fault and is not one. The bulk capacitor absorbs the current step when a servo starts, which is what otherwise dips the rail.

**Prove it**

Meter across the rail. **5 V ±0.25 V, steady.** Nothing else is connected yet, so anything else is the supply or the wiring, not the firmware.

## 10 Stage 6 — one servo

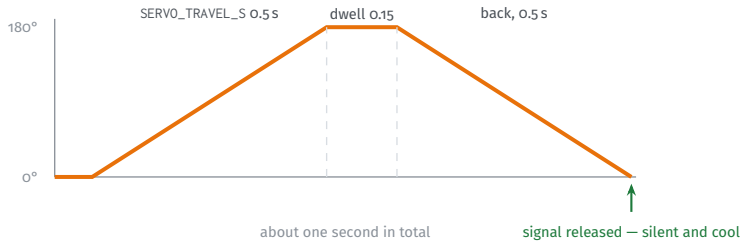
Do one, entirely, before the other two.




### Prove it

```
sudo systemctl restart snapcare-dispenser
curl -X POST http://localhost/api/drop -d '{"tray":1}'
```

One sweep: rest at 0°, out to 180°, back to 0°, about a second in total. Then the signal to that pin is **cut** rather than held — so the servo should go silent and cool between doses.




**Check which driver you got.** `servoBackend` in the status, with `servoSteady` saying whether its timing can be trusted:

```
curl -s http://localhost/api/status | grep -i servo
```

| Backend | Verdict | What it means |
|---|---|---|
| `pigpio` | what you want | Pulses timed by the DMA controller, immune to CPU load |
| `rpigpio` | jittery fallback | Pulses timed from a Python thread; every kernel preemption is a visible twitch and an audible buzz. Reports `servoSteady: false`. Fix with `sudo systemctl enable --now pigpiod` |
| `mock` | nothing moves | No servo library present at all |

## 11 Stage 7 — the other two servos

| Tray | GPIO | Header pin |
|---|---|---|
| 1 | 17 | 11 |
| 2 | 27 | 13 |
| 3 | 22 | 15 |

`INSTALLED_TRAYS = 3` in `config.py` is the only thing that says three. The map lists all eight, so scaling up is one number and some servos. Trays 4–8 are reserved at GPIO 5, 6, 13, 19 and 26 — already checked for clashes, so a fourth hopper is a plug and a number.

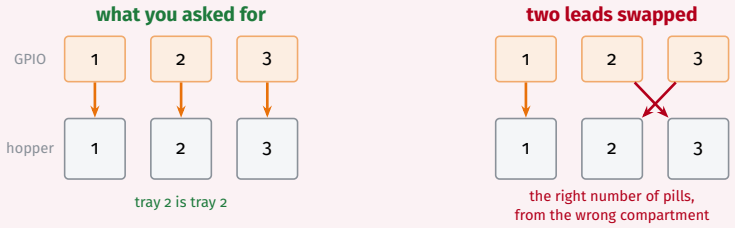
**Prove it**

Prove each one *individually*, and watch which hopper actually moves:

```
for t in 1 2 3; do curl -X POST http://localhost/api/drop -d "{\"tray\":$t}"; sleep 2; done
```

**This is the one worth doing carefully**

Two trays on one pin gives the right number of pills from the wrong compartment, and **nobody notices that until the wrong medication has been taken for a week.** The firmware refuses a duplicate pin at startup, but it cannot check that the wire you plugged into GPIO 27 goes to the hopper you are calling tray 2.




**Only one servo is ever driven at a time**, enforced by a lock — three doses owed at the same minute still move one horn at a time. Eight SG90s starting together is a current step that reboots the Pi with a compartment open.

## 12 Stage 8 — mechanical fitting and trim

Fit the hoppers with the servos at their **resting 0°**, not wherever the horn happened to stop. Park each one there first, then mount the horn:

```
sudo python3 -m snapcare_dispenser --drop 1     # sweeps, and ends at rest
```




If a tray does not quite reach its stop, or buzzes at the end of the stroke because it is being driven past it, trim *that tray alone*:

```
SERVO_TRIM = {3: (520, 2450)}    # config.py -- overrides only tray 3
```

**Buzzing at the end of a stroke is the one to act on**

It is the servo fighting a mechanical stop it has already reached, and it is what burns one out over a few weeks.

The other two knobs, if a full hopper is slower than an empty one:

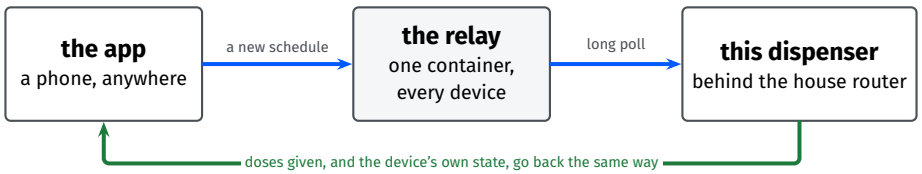
```
SERVO_TRAVEL_S = 0.5     # one traverse; open-loop, nothing reports arrival
SERVO_DWELL_S  = 0.15    # pause at the far end before returning
```

`SERVO_TRAVEL_S` must cover the *slowest* case — a cold servo under the weight of a full tray — because nothing on an SG90 reports position and the firmware is simply waiting out the clock.

## 13 Stage 9 — cloud sync, if you want it

Skip this stage entirely and everything still works: the schedule arrives from the app over the home WiFi or off a photographed paper sheet, and the device runs it from the SD card against the DS3231. That is the whole machine, and it is a reasonable way to ship one.

What this stage adds is that a schedule edited in the app reaches the device **without anybody being in the house** — which is the normal case, because the person managing a prescription is usually not the person taking it.




the relay is *not* a part in the bill of materials:
one container, deployed once, serving every dispenser you build

See `../cloud_relay/README.md`. Then name it here:

```
curl -X POST http://localhost/api/cloud \
     -H 'Content-Type: application/json' \
     -d '{"relayUrl":"https://snapcare-relay.fly.dev"}'
```

or type it into the device's own console, or set `SNAPCARE_RELAY_URL` in the systemd unit if you are building more than one.

**Prove it**

```
curl -s http://localhost/api/status     # the "cloud" object
```

`state` should reach `connected` within a few seconds, with `lastError` empty and `reportedRev` at 1 or more — that last one is the device having published its own state, which is what creates its mailbox.

Then prove the direction that matters, from the relay's demo app: open `https://<your relay>/lab`, pair with this device's code, and send it a schedule. It should appear in `payload` here within a second or two.

**If it does not:**

| `state` | What it means |
|---|---|
| `unpaired` | No relay URL. The POST above did not take, or was sent to the wrong host |
| `unsupported` | `python3-cryptography` is missing. `sudo apt install python3-cryptography` |
| `retrying` | The relay is not answering. `lastError` says why — usually DNS or a firewall |
| `offline` | It was reachable and has not been for fifteen minutes |
| `off` | Switched off — `--no-cloud`, or `SNAPCARE_CLOUD=0` |

**The test nobody does: unplug the internet, not the device**

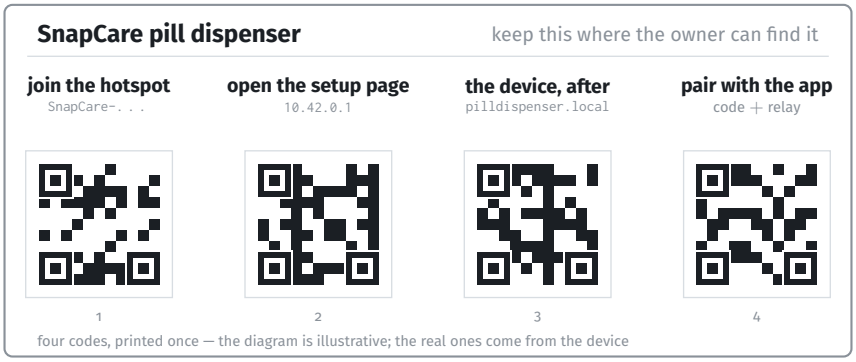
Pull the router's WAN cable, leave the dispenser on its LAN, and check that a scheduled dose still comes out on time. It must. A build that has just gained a cloud feature is exactly when a dependency on it gets introduced by accident, and the symptom is a machine that works perfectly in every test anyone runs at a desk.

## 14 Stage 10 — the label

The setup code is printed once and never changes. It is derived, not generated: HMAC-SHA256 over the SoC serial in `/proc/cpuinfo`, so it survives a reimage and differs per board. The pairing code on it comes from the same derivation, so it is permanent in the same way.

```
sudo python3 -m snapcare_dispenser --show-qr      # scannable, in the terminal
```

or print the proper one from `http://pilldispenser.local/label`.




**The pairing code is a credential**

Anyone who can read it can send this device a schedule, in the same way anyone who can read the WiFi password can join the hotspot. Put the label where the owner can find it and a visitor cannot — inside a battery door, or on the underside.

**The one part of the label that is not permanent**

The pairing QR encodes *your relay's address* alongside the code, because that is what makes a phone camera able to open it. Move to a different relay and that QR points at the old one. The code printed beside it in text still works — it is derived from the board, not from the relay — so the fix is to reprint, not to re-pair. If you expect to move relays, print the label with no relay configured: the QR then carries the code alone, which the app can read and a plain camera cannot.

## 15 The whole pin map, in one place

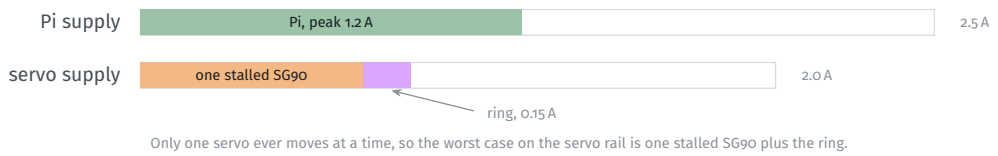
| | Signal | GPIO (BCM) | Header pin | Notes |
|---|---|---|---|---|
| | Servo tray 1 | 17 | 11 | signal only — **not** power |
| | Servo tray 2 | 27 | 13 | |
| | Servo tray 3 | 22 | 15 | |
| | *Servo trays 4–8* | *5, 6, 13, 19, 26* | *29, 31, 33, 35, 37* | reserved, unfitted |
| | WS2812 ring data | 18 | 12 | through 330 Ω at the ring; must be 18/12/21/10 |
| | Piezo buzzer | 23 | 16 | through 100 Ω |
| | Confirm button | 24 | 18 | other leg to GND; internal pull-up |
| | DS3231 SDA | 2 | 3 | the module's own pull-ups are enough |
| | DS3231 SCL | 3 | 5 | |
| | DS3231 VCC | — | 1 (3.3 V) | not 5 V |
| | Ground | — | 6, 9, 14, 20, 25, 30, 34, 39 | at least one to the servo supply GND |

Run `pinout` on the Pi for the full header. Everything above is BCM numbering, which is what pigpio and RPi.GPIO both use.

## 16 Power budget

Two supplies, and the split is the point.

| Rail | Load | Draw |
|---|---|---|
| Pi supply (5 V 2.5 A) | Pi 3B with WiFi active | ~0.4 A idle, ~1.2 A peak |
| Servo supply (5 V 2 A+) | one SG90, moving | ~0.2 A typical, ~0.7 A stalled |
| Servo supply | WS2812 ring, 12 px at brightness 60 | ~0.15 A (≈0.7 A at full white) |




Figures for the servo and the ring are the ones the firmware was designed around; the Pi's are the usual published ranges. Meter your own rail if you change the ring size or fit more than three trays.

## 17 Commissioning checklist

Run this before the device goes anywhere near real medication.

```
curl -s http://localhost/api/status
```

- ☐ `pigpio` backend, not `rpigpio` or `mock`
- ☐ Clock trusted, source `rtc` or `ntp`
- ☐ Ring **and** buzzer listed as fitted
- ☐ Button backend `pigpio`
- ☐ Three trays reported, matching the three that physically exist
- ☐ Each `POST /api/drop` moves the hopper with the number you asked for
- ☐ Servos silent and cool 5 s after a drop
- ☐ Pull the power mid-schedule, restore it: no second dose, and the log survives
- ☐ Unplug the Pi's network, reboot: the DS3231 still vouches for the time
- ☐ The printed label's QR joins the hotspot and opens the setup page
- ☐ **Type the WiFi password wrong on purpose.** The setup network comes back within a few seconds and the page you re-open says the password was not accepted
- ☐ Type it right: the ring stops being cyan and the device answers at `pilldispenser.local` from a phone back on the house WiFi
- ☐ Change the router's password, reboot the device, wait: the setup hotspot appears on its own rather than the device sitting there unreachable
- ☐ Power the router and the device off together and bring only the device back for a few minutes first. It raises its hotspot, then finds the router by itself once that returns — no hands

**And if you did Stage 9:**

- ☐ `cloud.state` is `connected`, `lastError` empty
- ☐ A schedule sent from the app appears in `payload` within a few seconds
- ☐ A dose given here appears at the app end within a few seconds
- ☐ **Cut the internet, keep the LAN: a scheduled dose still comes out on time**
- ☐ Restore it: `state` returns to `connected` on its own, with nothing restarted
- ☐ The doses given while it was offline arrive at the app afterwards
- ☐ Send the same schedule revision twice: the second is ignored, not re-applied

**Three of these are worth doing properly rather than ticking.**

**The power cut.** A cut at 08:30 after the 08:00 dose is exactly the case that dispenses twice without persisted state, and it is invisible in a demo.

**The internet cut.** Not the same test as unplugging the network, and it is the one that catches a dependency on the relay introduced by accident — because everything still passes at a desk with working internet.

**The repeated revision.** The relay refuses it, so the only way to see the *device's* own check is to test it here. That check is what stands between a rolled-back relay and last month's prescription being applied as if it were new.

## 18 When it does not work

| Symptom | Look at |
|---|---|
| Servo twitches or ignores commands | Common ground. The Pi's GND and the servo supply GND must be joined |
| Servo hums between doses | You are on the `rpigpio` fallback — `sudo systemctl enable --now pigpiod` |
| Servo buzzes at the end of a stroke | Being driven past its mechanical stop. Trim it in `SERVO_TRIM` before it burns out |
| Pi reboots when a servo moves | Servos are on the Pi's 5 V pin, or the rail has no bulk capacitor |
| Ring never lights, DMA error in the log | Data line is not on GPIO 18/12/21/10 |
| First pixel is the wrong colour | 330 Ω missing, or at the Pi end instead of the ring end. If it persists, the ring's 5 V logic threshold ($0.7 \times V_{DD} = 3.5$ V) is above the Pi's 3.3 V output — shorten the lead, or drop the ring's supply to ∼4.5 V |
| Ring flickers only when a servo moves | pigpio and the ring contending for PWM. Start pigpiod with `-t 0` to pin it to PCM |
| Buzzer makes a tone but not *this* tone | It is an active buzzer with its own oscillator. Use a passive element |
| One press logs several doses | Debounce — check `journalctl` says the `pigpio` button backend, not `poll` |
| Tray stays shut, nothing dispenses | The clock is not trusted. `sudo hwclock -r`, check the coin cell, then `POST /api/clock` |
| Clock right at shutdown, wrong at boot | `fake-hwclock` is back, overwriting the RTC. The installer disables it |
| `pilldispenser.local` does not resolve | avahi, and the hostname. The web app's `DEFAULT_HOST` is that exact string |
| Ring is cyan and stays cyan | The device is on its own setup hotspot waiting to be put on a network. Join `SnapCare-. . .` and the page opens itself |
| Setup page says "did not hand out an address" | The password was right and DHCP was not. Usually a router that needs restarting, or a MAC filter |
| The setup hotspot keeps disappearing for half a minute | That is the unattended retry finding a saved network worth trying. It stops once one works; raise `HOTSPOT_RETRY_S` if it is in the way of a bench session |
| A schedule POST answers 421 | The `Host` header is not a name this device answers to. Use `pilldispenser.local` or the IP; `SNAPCARE_EXTRA_HOSTS` adds a router's own domain |
| Setup page never leaves "waiting…" | The phone has not gone back to the house WiFi. Switch it over; the page follows. If the device's own network reappeared, the password was refused |
| A schedule is refused by tray number | That tray is not fitted. `INSTALLED_TRAYS`, or `SNAPCARE_TRAYS=8` |
| Cloud `state: unsupported` | `sudo apt install python3-cryptography`. Everything else still works meanwhile |
| Cloud `state: retrying`, DNS in `lastError` | The Pi has no working resolver. It joined the WiFi but got no DNS, or the router blocks it |
| Cloud connects, but no schedule ever arrives | The app is paired to a different code. Compare `cloud.deviceId` here with the one the app shows — they must match exactly |
| The app shows the device as never seen | Nothing has written to the mailbox. Check `reportedRev` here is 1 or more |

| Symptom | Look at |
|---|---|
| A schedule sent from the app is refused | `cloud.lastError` says why, in the same words the LAN API would use. Usually a tray that is not fitted |
| Cloud fine, but doses take minutes to appear | `SNAPCARE_CLOUD_IDLE_S` is set. That is the cheap mode working as asked |

**Nothing here needs the hardware to reproduce the *logic* side of it:**

```
make            # the full suite, no board, no servos, no network
make run        # the whole firmware on a laptop, mock servos, port 8099
make -C .. demo # the same, plus a relay and the demo app on localhost:8088
```

`make run` is the real firmware with the servo driver swapped for one that records what it was told — the API, the scheduler, the web console and the persistence all work, and nothing moves. `make demo` adds the other end: a relay and a browser page that pairs with it, so the whole cloud path can be exercised, including the failure cases, without a Raspberry Pi anywhere.